

Grupo

1201560 – Reinaldo Reis

1240160 – Nuno Castro

5 Blocked matrix processing

A implementação encontra-se [aqui](#).

Como executar

```
make init
make blocked_matrix_gcc_run
make blocked_matrix_llvm_run
```

6 Analyzing and Evaluating

N = tamanho da matriz; BS = tamanho do bloco; Time_SEQ = tempo de execução sequencial; Time_A = tempo de execução alínea a; Time_B = tempo de execução alínea b.

Nota: Os testes foram realizados num computador com um processador Intel(R) Core(TM) Ultra 7 155H (1.40 GHz), dentro de uma máquina virtual Ubuntu com 8 GB de RAM e 8 núcleos. Foram efetuadas 20 iterações, tendo-se posteriormente calculado a média dos resultados.

GCC

N	BS	Threads	Time_SEQ	Time_A	Time_B
512	64	1	0,001641	0,159738	0,003864
512	64	2	0,002038	0,194809	0,002975
512	64	4	0,001796	0,193027	0,002941
512	64	8	0,001917	0,297613	0,004871

N	BS	Threads	Time_SEQ	Time_A	Time_B
512	128	1	0,00192	0,148775	0,003402
512	128	2	0,001628	0,181861	0,002775
512	128	4	0,001842	0,188157	0,003137
512	128	8	0,001749	0,289155	0,004872
512	256	1	0,00207	0,157593	0,003087
512	256	2	0,001768	0,196862	0,003432
512	256	4	0,001587	0,198844	0,002824
512	256	8	0,00202	0,275625	0,005142
1024	64	1	0,001968	0,148979	0,003259
1024	64	2	0,001931	0,203037	0,002649
1024	64	4	0,001576	0,201384	0,003144
1024	64	8	0,002088	0,294554	0,006115
1024	128	1	0,001857	0,15997	0,003917
1024	128	2	0,001728	0,17497	0,002836
1024	128	4	0,002278	0,194852	0,002932
1024	128	8	0,002201	0,285795	0,004784
1024	256	1	0,002014	0,15313	0,003615
1024	256	2	0,002046	0,203572	0,003429
1024	256	4	0,002279	0,18249	0,003118
1024	256	8	0,00162	0,30115	0,005144
2048	64	1	0,001835	0,152802	0,004062
2048	64	2	0,001899	0,195735	0,002822
2048	64	4	0,002335	0,187414	0,003262
2048	64	8	0,002	0,299703	0,005967
2048	128	1	0,001887	0,158423	0,004507
2048	128	2	0,001911	0,198996	0,004303
2048	128	4	0,00192	0,184998	0,003799
2048	128	8	0,001776	0,296616	0,005165

N	BS	Threads	Time_SEQ	Time_A	Time_B
2048	256	1	0,001818	0,148992	0,003481
2048	256	2	0,001614	0,191794	0,003453
2048	256	4	0,001904	0,197177	0,003523
2048	256	8	0,001983	0,319074	0,006443

LLVM

N	BS	Threads	Time_SEQ	Time_A	Time_B
512	64	1	0,00154	0,013448	0,002479
512	64	2	0,002015	0,672506	0,009241
512	64	4	0,001601	0,50689	0,00672
512	64	8	0,001871	0,579332	0,009715
512	128	1	0,00166	0,014452	0,001807
512	128	2	0,002188	0,602177	0,009496
512	128	4	0,001751	0,491682	0,00565
512	128	8	0,001963	0,51786	0,005945
512	256	1	0,001914	0,012398	0,002202
512	256	2	0,001542	0,625795	0,008779
512	256	4	0,001911	0,493271	0,006264
512	256	8	0,001999	0,505148	0,00597
1024	64	1	0,001705	0,01594	0,003029
1024	64	2	0,001769	0,633144	0,010172
1024	64	4	0,001931	0,494828	0,005744
1024	64	8	0,001894	0,501872	0,006732
1024	128	1	0,002065	0,013241	0,002751
1024	128	2	0,001817	0,613801	0,012198
1024	128	4	0,001852	0,498725	0,00549
1024	128	8	0,002102	0,497378	0,007393
1024	256	1	0,001627	0,012532	0,001674

N	BS	Threads	Time_SEQ	Time_A	Time_B
1024	256	2	0,001978	0,596035	0,00912
1024	256	4	0,001878	0,496183	0,006795
1024	256	8	0,001704	0,557857	0,00767
2048	64	1	0,001715	0,012854	0,002344
2048	64	2	0,002381	0,614902	0,011086
2048	64	4	0,001955	0,505251	0,005937
2048	64	8	0,001733	0,526198	0,007139
2048	128	1	0,001701	0,012776	0,001816
2048	128	2	0,001691	0,562776	0,007337
2048	128	4	0,002087	0,504917	0,005848
2048	128	8	0,00183	0,542183	0,006316
2048	256	1	0,002204	0,014849	0,002276
2048	256	2	0,001754	0,595239	0,010189
2048	256	4	0,001805	0,51991	0,007964
2048	256	8	0,001925	0,61328	0,007779

a. Compare the execution time of the different approaches to parallelize program

A análise dos tempos de execução para as três abordagens (sequencial - `Time_SEQ`, paralela por elemento - `Time_A`, e paralela por blocos - `Time_B`) revela insights importantes sobre o modelo de `tasking` do OpenMP e a gestão da granularidade.

1. Versão Paralela por Elemento (`Time_A`) vs. Sequencial (`Time_SEQ`):

- **Observação:** A versão que paraleliza por elemento (`Time_A`) é consistentemente e drasticamente mais lenta do que a versão sequencial (`Time_SEQ`) em todos os testes, independentemente do tamanho da matriz (`N`) ou do número de `threads`. Os valores de `Time_A` são ordens de magnitude maiores que `Time_SEQ`.
- **Análise:** Este resultado deve-se ao problema da `Fine-grain Parallelism` e ao consequente `overhead` de criação e gestão de tarefas.

- Na abordagem paralela por elemento, cada célula da matriz é tratada como uma tarefa individual, o que resulta em centenas de milhares de tarefas para matrizes de tamanho médio.
- Cada tarefa realiza um trabalho computacional mínimo, mas o custo associado à sua criação, sincronização e gestão é elevado.
- Assim, o tempo total é dominado pelo custo de coordenação do paralelismo e não pelo cálculo propriamente dito, o que conduz a uma degradação acentuada do desempenho.

2. Versão Paralela por Blocos (`Time_B`) vs. Sequencial (`Time_SEQ`):

- **Observação:** A versão paralela por blocos (`Time_B`) é significativamente mais rápida que a versão paralela por elemento (`Time_A`), aproximando-se muito dos tempos sequenciais, embora em muitos casos continue ligeiramente mais lenta.
- **Análise:** Este resultado deve-se ao problema do `overhead`, que é mitigado pela utilização de **Coarse-grain Parallelism**.
 - Na abordagem paralela por blocos, cada bloco da matriz é tratado como uma tarefa, em vez de criar uma tarefa por célula. Cada tarefa executa loops sequenciais sobre os elementos do bloco, concentrando uma quantidade significativa de trabalho computacional entre os eventos de sincronização.
 - Isto reduz drasticamente o número de tarefas criadas e aumenta a quantidade de trabalho útil por tarefa, melhorando a relação computação/`overhead`.
 - Contudo, a operação em questão (`M[i][j] = (M[i][j-1] + M[i-1][j] + M[i][j+1] + M[i+1][j])/4.0;`) envolve dependências de dados (`depend(in: A[i][j-1], A[i-1,j]) depend(out: A[i][j])`), o que limita o paralelismo. Nem todas as tarefas podem ser executadas simultaneamente, ou seja o cálculo progride numa sequência parcial.
 - Para matrizes relativamente pequenas ou operações simples por célula/bloco, o `overhead` de criar e sincronizar tarefas ainda pode fazer com que a versão paralela por blocos não supere a versão sequencial. A vantagem do paralelismo torna-se mais evidente apenas em matrizes maiores ou com operações mais complexas por bloco.

3. Comparação entre as Abordagens Paralelas (`Time_A` vs. `Time_B`):

- **Observação:** A versão paralela por blocos (`Time_B`) apresenta um desempenho muito superior à versão paralela por elemento (`Time_A`) em todos os testes, sendo centenas de vezes mais rápida.
- **Análise:** Esta diferença drástica deve-se à importância da granularidade no paralelismo.

- A divisão em blocos é eficaz quando a computação por célula é simples e o overhead de criar uma tarefa por célula seria elevado.
- Ao agrupar o trabalho em blocos, reduz-se o número de tarefas a gerir e, consequentemente, o overhead de sincronização e agendamento.
- Esta abordagem permite um aproveitamento mais eficiente dos recursos paralelos, evidenciando a vantagem do `Coarse-grain Parallelism` sobre o `Fine-grain Parallelism`.

b. Compare the strategy used by both implementation for the execution of tasks by the threads

Para facilitar a análise dos tempos, podemos inicialmente fazer um cálculo simples:

$(\text{tempo do LLVM} - \text{tempo do GCC}) / \text{tempo do GCC} * 100$

Desta forma, obtemos a percentagem de diferença de desempenho entre o LLVM e o GCC: onde se o valor for maior que 0, o GCC teve melhor desempenho; se for menor que 0, o LLVM teve melhor desempenho.

N	BS	Threads	Time_A	Time_B
512	64	1	-6,154783668	-91,58121424
512	64	2	-1,128557409	245,2130035
512	64	4	-10,85746102	162,6005688
512	64	8	-2,399582681	94,65950748
512	128	1	-13,54166667	-90,28600235
512	128	2	34,3980344	231,1193714
512	128	4	-4,940282302	161,3147531
512	128	8	12,23556318	79,09425741
512	256	1	-7,536231884	-92,1328993
512	256	2	-12,78280543	217,8851175
512	256	4	20,41587902	148,0693408
512	256	8	-1,03960396	83,27365079
1024	64	1	-13,36382114	-89,30050544
1024	64	2	-8,389435526	211,8367588

N	BS	Threads	Time_A	Time_B
1024	64	4	22,52538071	145,7136615
1024	64	8	-9,291187739	70,38369874
1024	128	1	11,2008616	-91,72282303
1024	128	2	5,150462963	250,8035663
1024	128	4	-18,70061457	155,9506703
1024	128	8	-4,497955475	74,03313564
1024	256	1	-19,21549156	-91,81610396
1024	256	2	-3,323558162	192,7883009
1024	256	4	-17,59543659	171,8959943
1024	256	8	5,185185185	85,24223809
2048	64	1	-6,539509537	-91,58780644
2048	64	2	25,38177988	214,1502542
2048	64	4	-16,27408994	169,5908523
2048	64	8	-13,35	75,57315075
2048	128	1	-9,856915739	-91,93551441
2048	128	2	-11,51229723	182,8076946
2048	128	4	8,697916667	172,9310587
2048	128	8	3,040540541	82,78953259
2048	256	1	21,23212321	-90,03369308
2048	256	2	8,674101611	210,3532957
2048	256	4	-5,199579832	163,676798
2048	256	8	-2,924861321	92,20619668

Nota: Valores em verde indicam que o LLVM foi mais rápido; valores em azul indicam que o GCC foi mais rápido.

1. Observação Geral:

- Para o Time_A, os resultados mostram que o LLVM tende a apresentar um desempenho ligeiramente superior ao GCC, embora essa diferença seja geralmente pequena, situando-se entre 1% e 10% na maioria dos casos. As discrepâncias mais elevadas observadas em alguns testes pontuais não configuram um padrão consistente e poderão dever-se a flutuações

momentâneas do sistema, como variações de carga ou interferências de outros processos. Assim, de forma global, o desempenho do LLVM e do GCC nesta abordagem é semelhante, com apenas uma vantagem marginal para o LLVM em situações específicas.

- Para o `Time_B`, o LLVM também tende a ser mais lento que o GCC, especialmente com múltiplas `threads`, onde pode ser 70% a 250% mais lento. Com 1 `thread`, o LLVM consegue ser significativamente mais rápido que o GCC (percentagens negativas de -89% a -92%).

2. Análise das Estratégias de Implementação do Runtime:

As diferenças de desempenho observadas, especialmente a superioridade do GCC em cenários com múltiplas threads na abordagem por blocos (`Time_B`), podem ser explicadas pelas distintas estratégias de implementação do runtime de OpenMP, ou seja o OpenMP define o comportamento das tarefas, mas não a sua implementação. Isso dá liberdade aos compiladores (como GCC e LLVM) para adotarem diferentes estratégias de agendamento de tarefas, que impactam diretamente o desempenho.

- **Breadth-first** : Implementações mais antigas ou simples, usam uma única fila de tarefas para toda a equipa de `threads`.
- **Work-Stealing** : Implementações mais recentes, como o LLVM, seguem uma abordagem em que cada thread possui a sua própria fila de tarefas, e as threads com menos tarefas roubam tarefas das filas de outras threads, equilibrando dinamicamente a carga e aumentando a eficiência do sistema.

Com base nestes conceitos, podemos analisar os resultados:

1. Superioridade do GCC em `Time_B` com Múltiplas Threads:

- Como já foi referido, neste problema as operações dependem de outras operações anteriores. Isto significa que o paralelismo é inerentemente limitado e estruturado, não existindo um grande número de tarefas independentes que possam ser executadas livremente a qualquer momento.
- Neste cenário, um agendador sofisticado como o `work-stealing` (LLVM) pode introduzir um `overhead` maior do que os seus benefícios. A complexidade de gerir múltiplas filas, verificar se há tarefas para roubar e sincronizar esse processo pode ser mais custosa do que uma abordagem de agendamento potencialmente mais simples que o GCC usa por defeito. Como o número de tarefas "executáveis" a cada momento é baixo devido às dependências, a capacidade do `work-stealing` de balancear a carga dinamicamente não é totalmente explorada, e o seu custo de gestão torna-se mais visível.
- Em resumo, para este problema específico e altamente dependente, a estratégia de agendamento do GCC parece ser mais eficiente ou ter menor overhead, resultando em tempos de execução significativamente melhores.

2. Superioridade do LLVM em `Time_B` com 1 Thread:

- Com apenas uma thread, não há paralelismo real, nem roubo de trabalho. A diferença de desempenho neste caso não se deve à estratégia de agendamento de tarefas, mas sim a outros fatores, como a eficiência do código sequencial gerado pelo compilador ou o overhead mínimo do runtime para configurar o ambiente de tasking, mesmo que não seja usado para paralelismo. O LLVM parece ser mais eficiente neste cenário de "falso paralelismo".

3. Desempenho Semelhante em `Time_A`:

- Na abordagem por elemento (`Time_A`), o desempenho é dominado pelo `overhead` massivo da criação de milhões de tarefas minúsculas. Ambas as implementações (GCC e LLVM) sofrem com este problema de granularidade excessivamente fina. Embora o LLVM mostre uma ligeira vantagem, a diferença é marginal em comparação com a degradação geral do desempenho. A abordagem é tão ineficiente que as subtilezas entre as estratégias de agendamento se tornam menos relevantes do que o problema fundamental do overhead. A pequena vantagem do LLVM pode dever-se a uma implementação ligeiramente mais otimizada do mecanismo de criação/enfileiramento de tarefas.